

Day 6 – Structs in Swift

◆ What is a Struct?

A **Struct** is a custom data type used to group related properties and functions together.

Example

```
struct Dog {  
    let name: String  
}  
  
let myDog = Dog(name: "Tommy")  
  
print(myDog.name)
```

◆ Struct with Multiple Properties

```
struct Student {  
    let name: String  
    let rollNo: Int  
}  
  
let myStudent = Student(name: "Adarsh", rollNo: 14)  
  
print(myStudent.name)  
print(myStudent.rollNo)
```

◆ Structs are Value Types

Structs in Swift are **Value Types**.

This means:

- A copy gets created when assigning to another variable
- Changing the copy does not affect the original value

Example

```
struct User {  
    var name: String  
}  
  
var user1 = User(name: "Adarsh")  
var user2 = user1  
  
user2.name = "Robin"  
  
print(user1.name)
```

```
print(user2.name)
```

Output

```
Adarsh  
Robin
```

◆ Mutating Functions

Struct methods cannot modify properties unless marked with `mutating`.

Example

```
struct Player {  
    var name: String  
    var score: Int  
  
    mutating func increaseScore() {  
        score += 23  
    }  
}
```

◆ Computed Properties

Computed properties calculate values dynamically.

Example

```
var level: String {  
    if score > 50 {  
        return "Pro"  
    } else {  
        return "Beginner"  
    }  
}
```

◆ Multiple Initializers

Structs can have multiple initializers.

Example

```
struct User {  
    var name: String  
    var age: Int  
  
    init(name: String) {  
        self.name = name  
    }  
}
```

```
        self.age = 0
    }

    init(name: String, age: Int) {
        self.name = name
        self.age = age
    }
}
```

◆ Real-World Example – Instagram Profile

```
struct Instagram {
    var username: String
    var followers: Int
    var following: Int
    var posts: Int
    var bio: String

    func showProfile() {
        print(username)
        print(bio)
        print(followers)
    }

    var accountType: String {
        if followers > 800 {
            return "Popular"
        } else {
            return "Normal"
        }
    }
}
```

🧠 Key Concepts Learned

- Structs group related data together
 - Structs are Value Types
 - Functions inside structs are called Methods
 - `mutating` keyword allows modifying properties
 - Computed properties calculate values dynamically
 - Initializers set default values during object creation
-

🎯 Important Interview Questions & Answers

1 What is a Struct in Swift?

A Struct is a custom data type used to store related properties and methods together.

2 What is the difference between Struct and Class?

Struct	Class
Value Type	Reference Type
Faster	Slightly heavier
Copies data	Shares reference
No inheritance	Supports inheritance

3 What are Value Types?

Value types create a copy when assigned to another variable.

Example

```
var user1 = User(name: "Adarsh")
var user2 = user1
```

Changing `user2` does not affect `user1`.

4 Why do we use `mutating` in Structs?

Because Structs are value types and Swift prevents modifying properties inside methods unless marked with `mutating`.

Example

```
mutating func increaseScore() {
    score += 1
}
```

5 What is a Computed Property?

A property that calculates and returns a value instead of storing it.

Example

```
var level: String {  
    return "Pro"  
}
```

What is an Initializer in Swift?

An initializer sets initial values when creating an instance of a Struct or Class.

Example

```
init(name: String) {  
    self.name = name  
}
```

Quick Revision

- Struct = Custom data type
 - Structs are Value Types
 - Methods = Functions inside Structs
 - `mutating` allows modification
 - Computed property = calculated value
 - Initializer = setup values during creation
-

Day 6 Summary

Today I learned:

- Struct creation
- Value type behavior
- Methods in structs
- Mutating functions
- Computed properties
- Multiple initializers
- Real-world modeling using structs